BLINDSIGHT

A Voice-First, Three-Tier Navigation and Awareness Companion for Blind and Low-Vision Users

Project Report — Science Exhibition / Academic Project Review

Category: Novel solution / indigenous design to a problem or challenge

Submitted by: **[Your Name]**, [Class & Roll No.]

Guide / Mentor: **[Mentor Name]**

Institution: **[School / College], [City]**

Academic Year: 2026–27

Source repository: github.com/devrift/BlindSight — MIT License

Certificate

*This is to certify that the project entitled "**BlindSight — A Voice-First, Three-Tier Navigation and Awareness Companion for Blind and Low-Vision Users**" is a bona fide record of project work carried out by **[Your Name]** during the academic year 2026–27, under my supervision. The work embodied in this report is original and has not been submitted elsewhere for the award of any prize or degree.*

>

***[Mentor Name & Designation]** — [Date & Place]*

Declaration

I declare that this report describes my own work; that every hardware measurement, field test and software result stated here is genuine; that all external sources are acknowledged in Section VII; and that the limitations of the system are stated openly in Section VI and Appendix E, as a mobility aid must never over-promise.

>

*— **[Your Name]***

Acknowledgements

*I thank my mentor **[Mentor Name]** for guidance and patience through every field-test failure; **[Institution]** for its facilities; my family and the volunteer testers whose blunt feedback — "the mic opens for one second then stops" — shaped versions 2 through 3.3; and the open-source communities whose work this project stands upon: Espressif, Ultralytics, the Flask and edge-tts maintainers, the React and Kotlin ecosystems, and the MDN contributors.*

Abstract

BlindSight is an affordable mobility aid for blind and low-vision pedestrians that speaks instead of showing. It combines three independent safety tiers: a network-free ultrasonic reflex on an ESP32 that commands "Stop" under 25 cm and steers the user between 25 and 45 cm; an offline edge-vision tier on a Raspberry Pi running a YOLOv8-nano ONNX model at roughly 1.5 seconds per frame; and a cloud companion — a voice-first progressive web app with a native Android shell — offering a wake word, conversational scene questions, personal memory and seven reply languages in human-sounding neural voices. One design rule governs the whole system: a mobility aid must fail loudly, never optimistically. The work was validated with up to 93 automated tests at its v3.3 delivery, continuous integration on every push, cloud-compiled Android builds, and iterative field trials on real Android, iPhone and Windows devices that exposed and fixed ten genuine defects. All code, firmware contracts and audit trails are public under the MIT licence.

Contents

- I. Title
- II. Nature of Work
- III. Rationale
- IV. Scientific Principles / Concepts

- V. Materials Used
- VI. Procedure / Description (including Future Scope)
- VII. References
- Appendix A — Companion HTTP API reference
- Appendix B — Voice command table
- Appendix C — Bill of materials
- Appendix D — Testing, CI and field-validation summary
- Appendix E — Honest limitations and bench-pending items

I. Title

BLINDSIGHT — A Voice-First, Three-Tier Navigation and Awareness Companion for Blind and Low-Vision Users.

II. Nature of Work

Novel solution / indigenous design to a problem or challenge ✔

(The project also applies basic principles of science — acoustics, radio propagation, neural networks — but its primary character is an original engineered response to an unmet accessibility need.)

III. Rationale

Blind and low-vision pedestrians still rely overwhelmingly on the white cane and, where affordable, guide dogs. Both report only what they touch: overhead branches, glass panels, knee-level hazards and moving objects remain invisible until contact. Commercial electronic travel aids exist but cost hundreds to thousands of dollars, placing them out of reach in the regions where most of the world's visually impaired people live. Meanwhile the smartphone a user already owns carries microphones, speakers and radios that remain almost entirely unused as a mobility aid.

BlindSight closes this gap with an affordable, voice-first companion built from commodity parts. It speaks what it sees, warns before contact, answers questions about a scene on demand, remembers personal facts such as "my bus stop is near the temple", and works in seven reply languages. Because it is a mobility aid, it obeys one rule above all others: fail loudly, never optimistically. Every tier reports its true state; no endpoint serves placeholder, stale or synthesised data as a live observation; a missing model, missing speech engine or missing API key produces an honest, spoken error instead of a confident lie.

IV. Scientific Principles / Concepts

- **Ultrasonic time-of-flight ranging (Tier 0).** Three HC-SR04 sensors emit 40 kHz pulses; distance equals the speed of sound multiplied by echo time, divided by two. A network-independent reflex on an ESP32 speaks "Stop" under 25 cm and steers left or right between 25 and 45 cm, so protection survives total Wi-Fi or cloud outage.
- **Low-latency device-to-device radio (ESP-NOW).** Connectionless ESP32-to-ESP32 frames carry head-camera JPEGs in checksummed, de-duplicated 220-byte chunks with stall recovery, keeping the camera wire-free.
- **Edge machine vision (Tier 1).** A YOLOv8-nano convolutional neural network, exported to ONNX, runs in an isolated subprocess on a Raspberry Pi 3B+ at roughly 1.5 s per frame, detecting objects with no internet connection.
- **Vision-language models (Tier 2).** A camera frame plus the user's spoken question reach a cloud model only when the user asks, producing natural-language scene descriptions and conversational answers.
- **Speech technology.** Browser Web Speech API provides recognition and the wake word; neural text-to-speech (Microsoft neural voices via edge-tts) is rendered once on the gateway so every device hears the same human voice; espeak-ng is the offline fallback; on iPhone, where no browser exposes continuous listening, the app routes dictation through the system keyboard and says so out

loud.

- **Progressive Web App plus native shell.** One voice-first codebase serves Android, iPhone and desktop; a Kotlin WebView shell adds a launcher icon, native microphone and camera permission prompts, and scoped self-signed-certificate trust for home networks.
- **Fail-closed engineering.** Bounded Wi-Fi waits, HTTP timeouts of at least 60 s, WAV format-and-spectrum verification of firmware audio clips, atomic state writes with quarantine of corrupt files, rate limiting, token-gated APIs, and an honest `/health` endpoint that reports the real state of every tier.

V. Materials Used

- ESP32 DevKit V1 head unit; ESP32-CAM head camera; 3× HC-SR04 ultrasonic sensors; PCM5102 I2S DAC with speaker; tactile button; buck converter and power bank.
- Raspberry Pi 3B+ with camera module (edge-vision gateway).
- Android phone and any modern browser (companion PWA); Android shell APK built in the cloud.
- Software: Python 3 with Flask and gunicorn (gateway generation), ONNX Runtime, YOLOv8n, edge-tts; TypeScript with React and Vite plus a Kotlin Jetpack Compose Android app (current generation); vanilla-JS service worker for offline use; PlatformIO and Arduino frameworks for firmware; GitHub Actions for continuous integration and cloud APK compilation.

VI. Procedure / Description

- **Reflex tier construction.** Firmware samples three ultrasonic sensors, maps distance bands (stop / steer / careful / clear) to pre-generated LittleFS voice clips played over I2S, and filters ESP-NOW frames by sender MAC so only the head camera is trusted. Three synchronised firmware entry points (PlatformIO, Arduino IDE, no-terminal installer) are held behaviourally identical by a written contract of twelve audited behaviours; the platform is pinned to espressif32@6.13.0 with a version guard that also compiles on core 3.x.
- **Edge-vision tier construction.** The Pi gateway captures frames, runs ONNX detection in a sandboxed subprocess, and exposes results over HTTPS with token-gated APIs. Per-request media is swept hourly and capped at 50 MB; conversation state is written atomically and quarantined if ever corrupt.
- **Companion tier construction.** A voice-first interface offers the wake word "hey BlindSight", hands-free and button modes, conversation memory (24 turns stored, 8 sent per request), up to 20 personal facts, and seven reply languages with script-following detection — Devanagari input answers in Hindi even against a saved preference. All local commands work with no internet.
- **Voice-quality iterations (v1 → v3.3).** Field trials on real devices exposed and fixed successive failure classes: recognition restarts with backoff so silence timeouts cannot kill the wake listener; finals-only results and an optional noise gate keep room noise out; robot browser voices were replaced by gateway-rendered neural voices; and version 3.3 removed the Android audio-focus defect in which a start-up beep stole the microphone from the speech recognizer about one second after it opened.
- **Cross-platform hardening.** iPhone cannot grant any web app continuous listening, so the app detects this and switches to an honestly spoken dictation flow; iOS audio is unlocked on first touch;

the Stop button silences neural audio as well as browser speech; token entry moved from browser popups (invisible in home-screen apps) to an inline settings field; notch, safe-area and soft-keyboard layout traps were fixed; a native Android shell now provides launcher presence and proper permission prompts.

- **Testing and validation.** At the v3.3 delivery the repository carried 93 automated tests covering fail-closed vision contracts, API contracts, memory integrity, WAV anti-fake gates and mobile platform contracts; continuous integration ran lint, JavaScript syntax checks, WAV verification and the test suite on every push, while a second workflow compiled the Android APK on cloud runners. Firmware copies were audited line-by-line against their twelve-behaviour contract. Field validation ran on Android, iPhone and Windows across multiple rounds.
- **Deployment.** One-click starters (Windows batch file; Pi shell script that generates self-signed HTTPS automatically), a systemd service with gunicorn, token-based access control, rate limiting, and plain-English quick-start and install guides make the system usable by non-programmers.
- **Current code generation.** The companion is being re-expressed as a TypeScript React PWA with a Kotlin Compose Android app, keeping the same tier architecture, firmware and safety contracts while modernising the client stack.
- **Future scope.** A chip-level always-on wake word on the ESP32; bench certification of the ultrasonic thresholds; on-device speech and vision models for full offline operation; more reply languages; a signed Play Store release; and integration of smarter canes or haptic feedback bands.

VII. References

- Espressif ESP-IDF and ESP-NOW documentation; esp32-camera driver reference (SCCB pin naming and version guards).
- Ultralytics YOLOv8 and ONNX Runtime model-export documentation.
- MDN Web Docs: Web Speech API, Service Workers, Secure Contexts; W3C WAI-ARIA authoring practices for accessible live regions.
- edge-tts project documentation (Microsoft neural voices); OpenRouter API documentation (vision-language model access).
- Flask and gunicorn deployment guides; React, Vite and Kotlin Jetpack Compose documentation; GitHub Actions documentation.
- Project's own engineering records: `docs/REPO_AUDIT.md` (full audit trail), `docs/ANDROID_APP.md` (Android shell guide), `firmware/FIRMWARE_VARIANTS.md` (three-copy firmware contract).

Appendix A — Companion HTTP API Reference

Route	Method	Auth	Purpose
`/`	GET	–	Voice-first PWA shell (screen-reader first-class)
`/health`	GET	–	Honest tier status, open issues, version; 503 when degraded
`/api/config`	GET	–	Whether a token is required; frame freshness limit
`/api/prefs`	GET / POST	POST: token	Reply language and voice preference plus options
`/api/tts/voices`	GET	–	Neural voice catalogue for the current language
`/api/chat`	POST	token	Prompt plus optional camera frame; reply with audio URL; 429 when rate-limited
`/api/latest-frame`	GET	token	Freshest camera JPEG (≤30 s); 404 when none — never a placeholder
`/analyse`	POST	–	Multipart image through Tier 1; summary plus audio
`/audio/<id>`	GET	–	Rendered reply audio (neural MP3 preferred, espeak WAV fallback)
`/sw.js`	GET	–	Service worker with root scope header

Appendix B — Voice Command Table

Say	Effect	Works offline
hey BlindSight, question	Wake word then one spoken question	No
describe / what is around me	Forces a camera-frame description	Vision tier only
remember that …	Stores a personal fact (max 20)	Yes
what do you remember / forget …	Recalls or removes facts	Yes
language hindi / spanish …	Changes reply language and voice	Yes
read status	Speaks the live health summary	Yes
clear chat / start over	Resets conversation history	Yes
stop / help	Silences the assistant / reads commands	Yes

Appendix C — Bill of Materials

Component	Tier	Role
ESP32 DevKit V1	0	Ultrasonic reflex, clip playback, ESP-NOW receiver
3× HC-SR04	0	Centre, left and right distance sensing

Component	Tier	Role
PCM5102 DAC + speaker	0	Voice clip output
ESP32-CAM	0→1	Head-camera JPEG uplink over ESP-NOW
Raspberry Pi 3B+	1	Gateway, edge ONNX vision, neural TTS rendering
Pi camera module	1	Scene frames for detection and cloud questions
Android phone / browser	2	Companion PWA, wake word, memory, languages
Android shell APK	2	Launcher icon, native permissions, cert trust

Appendix D — Testing, CI and Field-Validation Summary

- 93 automated tests at the v3.3 delivery: fail-closed vision contracts, HTTP API contracts, companion memory integrity, WAV format-and-spectrum anti-fake gates, PWA shell accessibility contract, mobile platform contract, repository hygiene guards.
- Continuous integration on every push: lint, JavaScript/TypeScript build checks, WAV verification, test suite; a separate workflow compiles the Android APK on cloud runners and publishes it as a downloadable artifact.
- Firmware contract audits: all three main-firmware copies and both camera copies verified line-by-line against twelve audited safety behaviours (bounded Wi-Fi wait, callback version guard, MAC filtering, audio-path correctness, timeouts, stall recovery, channel guards, aliasing-safe header parsing, sample-rate re-assertion, stop-band mapping, modem-sleep disable, SCCB spelling).
- Field rounds on real hardware: Android (wake word, hands-free, install), iPhone (dictation flow, audio unlock, home-screen install), Windows localhost testing; each round's defects were fixed and pinned by regression tests.

Appendix E — Honest Limitations and Bench-Pending Items

- No wake word on iPhone or iPad: Apple grants no web app continuous listening; the supported flow is spoken dictation via the system keyboard.
- Ultrasonic band thresholds (25 / 45 / 100 cm) and the ~1.5 s Tier-1 frame latency are measured-class values awaiting a formal bench session.
- Browser wake word needs a Chromium-class browser, network access, and an open page; a chip-level always-on wake word is the intended hardware answer.
- Neural voices require outbound internet on the gateway; offline operation falls back to espeak-ng or browser voices.
- Self-signed home HTTPS shows a one-time certificate warning on phones (the Android shell trusts the saved gateway host silently).
- ESP-NOW range is room-scale; the JPEG uplink is best-effort with stall recovery, not a video stream.
- The distributed Android APK is debug-signed; a Play Store release needs an individual signing key and developer account.